Exploration d'une extension PHP inconnue - Rapport

Realisé par: Aicha Nait Dayaa

Encadré par by: Mr. Abdelmajid Bendrif
Mr. Hamza Bahlaouane

Date: February 23, 2026

1 Exploration d'une extension PHP inconnue

1.1 Objectif

Identifier l'extension PHP responsable d'ajouter la fonction **/VoidLabs/canYouSeeMe()** dans un conteneur Docker.

1.2 Étapes Suivies

1. **Lancement du conteneur et Exécution du script**

```
root@2ea1cf64ab9f:/usr/src/myapp# php app.php
-=-=-=-=-=-=-=-=-=-=
Hello You!
 Find the the extension (.so) path and how it was loaded.-=-=-=-=-=-=-=-=-=-=
root@2ea1cf64ab9f:/usr/src/myapp# cat app.php
<?php

echo str_repeat("-=", 10) . "\n";

\VoidLabs\canYouSeeMe();

echo str_repeat("-=", 10) . "\n"; root@2ea1cf64ab9f:/usr/src/myapp#
root@2ea1cf64ab9f:/usr/src/myapp#
```

La sortie montre qu'on doit trouver l'extension .so qui est responsable d'avoir ajouté la fonction **/Void/canYouSeeMe()**.

2. **Localiser les extensions chargées**

On cherche le dossier où PHP stocke ses extentions, par la commande :

```
        php -i | grep extension_dir
```

```
extension_dir => /usr/local/lib/php/extensions/no-debug-non-zts-20220829 => /usr
/local/lib/php/extensions/no-debug-non-zts-20220829
```

Et après on liste les fichier .so présents par :

```
        ls /usr/local/lib/php/extensions/no-debug-non-zts-20230831/ |
            grep so
```

```
root@2ea1cf64ab9f:/usr/src/myapp# ls /usr/local/lib/php/extensions/no-debug-non-
zts-20220829 | grep so
opcache.so
sample.so
sodium.so
```

Maintenant, on vérifie lesquels sont réellement chargés dans le runtime par :

```
        php -r "var_dump(extension_loaded('sodium'));"
        php -r "var_dump(extension_loaded('sample'));"
```

```
root@430d97f79848:/usr/src/myapp# php -r "var_dump(extension_loaded('opcache'));
"
bool(false)
root@430d97f79848:/usr/src/myapp# php -r "var_dump(extension_loaded('sodium'));"
bool(true)
root@430d97f79848:/usr/src/myapp# php -r "var_dump(extension_loaded('szmple'));"
bool(false)
root@430d97f79848:/usr/src/myapp# php -r "var_dump(extension_loaded('sample'));"
bool(true)
```

Les 2 extensions sample.so et sodium.so sont les seules extensions .so chargées. On inspecte donc les fonctions de chacune pour identifier laquelle expose /VoidLabs/canYouSeeMe()

3. **Vérification des fonctions des extensions**

   On vérifie alors la fonction de chacune des 2 extensions par la commande :

```
        php -r "print_r(get_extension_funcs('sample'));"

        php -r "print_r(get_extension_funcs('sodium'));"
```

```
root@430d97f79848:/usr/src/myapp# php -r "print_r(get_extension_funcs('sample'))
;"
Array
(
    [0] => VoidLabs\canYouSeeMe
)
root@430d97f79848:/usr/src/myapp# php -r "print_r(get_extension_funcs('sodium'))
;"
Array
(
    [0] => sodium_crypto_aead_aes256gcm_is_available
    [1] => sodium_crypto_aead_aes256gcm_decrypt
    [2] => sodium_crypto_aead_aes256gcm_encrypt
    [3] => sodium_crypto_aead_aes256gcm_keygen
    [4] => sodium_crypto_aead_chacha20poly1305_decrypt
    [5] => sodium_crypto_aead_chacha20poly1305_encrypt
```

1.3 Conclusion

Alors on peut déduire que l'extension responsable est sample.so, car c'est la seule parmi les deux qui expose la fonction canYouSeeMe dans le namespace /VoidLabs.

2 Création d'une extension

2.1 Étapes pour créer une extension similaire

1. **Préparation de l'environnement**

   PHP ne compile pas les extensions par défaut, donc on doit installer des outils qui permettent de transformer le code C en module PHP utilisable.
   - **outils de développement:** phpize, php-config
   - **outils de build:** make, autoconf

2. **Génération de la squelette de l'extension**

   PHP attend une structure particulière pour compiler les extensions. Le squelette fournit cette structure prête à l'emploi.

   - **Structure de l'extention:**

   - Config.m4 : fichier de configuration du build
   - extension.c : fichier principal des fonction de l'extension
   - extension.h : Fichier d'en-tete contenant les déclarations et macros

3. **Déclaration des fonctions**

   Définition des fonctions est faite dans le fichier .c, ces fonctions sont enregistrés aupres du moteur Zend

4. **Préparation du build**

   Pour compiler une extension .so, PHP doit savoir qu'il s'agit d'un module partagé. Cela est configuré dans le fichier Config.m4 en ajoutant la commande –enable-monext pour activer la compilation partagée

5. **Compilation de l'extension** Compilation via phpize :

```
phpize
./configure --with-php-config=/chemin/vers/php-config
make
make install
```

   Produit un fichier .so (Unix/Linux) ou .dll (Windows).

6. **Activation de l'extension**

   PHP ne charge pas automatiquement les .so dynamiques, il faut donc les déclarer dans le fichier de configuration comme ci :

```
extension=monext.so
```

2.2 Ressources

https://www.phpinternalsbook.com/php7/extensions_design/extension_skeleton.html
https://www.phpinternalsbook.com/php7/build_system/building_extensions.html

3 Intérêt des extensions PHP

3.1 Performance

Une extension PHP est écrite en C et compilée en code natif, ce qui la rend bien plus rapide que du PHP pur. Pour des opérations intensives comme la cryptographie, le traitement d'images ou le parsing de grandes quantités de données, le gain est significatif.

3.2 Accès bas niveau

Une extension peut s'interfacer directement avec des bibliothèques système, ce qui est tout simplement impossible depuis du PHP classique. Cela ouvre la porte à des fonctionnalités qui dépassent les limites du langage.